

Improving task breakdown comprehensiveness in agile projects with an Interaction Room

Simon Grapenthin^a, Steven Poggel^b, Matthias Book^{c,*}, Volker Gruhn^a

^a paluno – The Ruhr Institute for Software Technology, University of Duisburg-Essen, Gerlingstr. 16, 45127 Essen, Germany

^b adesso AG, Stockholmer Allee 20, 44269 Dortmund, Germany

^c Department of Computer Science, University of Iceland, Dunhagi 5, 107 Reykjavík, Iceland

ARTICLE INFO

Article history:

Received 29 November 2014

Revised 20 July 2015

Accepted 24 July 2015

Available online 14 August 2015

Keywords:

Scrum

Sprint planning meeting

Task breakdown

Interaction Room

ABSTRACT

Context: The planning, estimation and controlling mechanisms of agile process models rely significantly on a fixed set of tasks being established for each sprint. These tasks are created as refinements of product backlog items at the beginning of each sprint. However, a project team's understanding of the backlog items' business implications and dependencies may often not be deep enough to identify all necessary tasks this early, so in addition to the tasks defined in the beginning of the sprint, more necessary tasks might be discovered as the sprint progresses, making any attempt at progress estimation or risk management difficult.

Objective: We strive to enable software teams to achieve a deeper understanding of product backlog items, which should help them to identify a sprint's tasks more reliably and comprehensively, and avoid discovering the need for extra tasks during sprint execution.

Method: We introduced a project team in a medium-sized software development company to the Interaction Room method, which encourages interdisciplinary communication about key system design aspects among all stakeholders. We observed the team's conduct in the sprint planning meetings, and tracked early- vs. late-identified tasks across several sprints.

Results: Before the introduction of our method, the team used to discover on average 26% of a sprint's tasks not at the beginning of the sprint, but later during the course of the sprint. Using the Interaction Room in two separate projects, this ratio dropped to an average of 5% late-discovered tasks.

Conclusion: Our observations from these projects suggest that increased communication among all stakeholders of a project leads to a more reliable identification of the tasks to be performed in a sprint, and that an Interaction Room can provide appropriate guidance to conduct this team communication in a focused and pragmatic way.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

Scrum is the most popular agile software process model [1]. It structures a project into a series of iterations that last from one to four weeks. Each iteration comprises the steps of iteration planning (in the *sprint planning meeting*), implementing and testing a certain product increment (in a *sprint*), presenting the results to stakeholders (in the *sprint review*) and reflecting on one's work in order to optimize future sprints (in the *sprint retrospective*) [2]. Scrum defines three roles that together form the so-called Scrum team: The *product owner* is responsible for managing the product, defining and prioritizing product requirements. The *Scrum master* manages the

process. He or she protects the team from interferences, identifies and eliminates impediments and advocates Scrum within the organization. The *development team* consists of particular roles needed for the development (e.g. developer, tester and architect). The development team is a proper subset of the Scrum team, excluding the Scrum master and the product owner. The development team members manage themselves in terms of how they complete the work to be done in a sprint [3]. The definition of this work occurs in the sprint planning meeting that takes place at the beginning of each sprint. It is divided into two separate meetings: In *sprint planning meeting 1*, the team gathers all information needed to estimate and commit to the implementation of a set of prioritized *backlog items*. A backlog item (or item for short) is a functional requirement defined by the product owner. In *sprint planning meeting 2*, the developers are then supposed to define all *tasks* that need to be performed in order to design, implement and test the backlog items selected for the sprint. Tasks describe the team's planned activities that are necessary in order to

* Corresponding author. Tel.: +354 5254603.

E-mail addresses: simon.grapenthin@paluno.uni-due.de (S. Grapenthin), steven.poggel@adesso.de (S. Poggel), book@hi.is (M. Book), volker.gruhn@paluno.uni-due.de (V. Gruhn).

fulfill the functional requirements. For each sprint, the development team commits to a limited number of backlog items, based on the estimates of the items' complexity and the team's velocity (i.e. the number of work units that can be handled by the team in a sprint). The velocity is a measurement that becomes more precise the more iterations a team completes, because its members gain experience in estimating the system's complexity and their own performance [4].

Deriving tasks from backlog items is a complex task, however. The team needs to have detailed knowledge about a variety of business and technical aspects of the information system under construction, as well as an understanding of the process and component landscape it is being integrated into, and the data structures it should rely on. Often, slicing tasks out of a backlog item is not straightforward. For example, if the specification of a backlog item is written from a user's perspective, as is often the case, the developers additionally have to make the transfer from the user perspective to the technical perspective. A team's inability to identify all tasks initially at the time of sprint planning meeting 2 therefore not only means that some tasks are simply recognized later in the course of the sprint, but also that the initial tasks may not be properly understood, and that they certainly do not take into account any dependencies with late-identified tasks. Besides the challenge of creating a clean overall system design under these circumstances, it is also difficult to measure the progress within a sprint, since the number of completed tasks (the *burndown*) cannot be used as a reliable metric to ensure that the work planned for a sprint gets done in its allotted timebox [5].

In this article, we describe an approach that fosters more in-depth communication about user stories and tasks between team members. While it encourages team members to examine their system's process, data, integration and interaction aspects explicitly, it does not aim to produce comprehensive specifications. Our hypothesis is that this pragmatic approach helps team members to understand the user stories better, and thus achieve a more comprehensive task breakdown. To test this hypothesis, we conducted a case study in two industry projects in collaboration with a medium-sized software company, where a Scrum team felt that their task breakdown was not as effective as it should be because many tasks were only identified over the course of the sprint, and not (as desired) in the initial sprint planning meeting 2. We therefore introduced the team to the so-called Interaction Room (an approach described in detail in [6, 7]) to support a more in-depth backlog item analysis and task breakdown during sprint planning meeting 2, and observed the team's performance.

In the following sections, we first give an overview of the project situation we initially encountered (Section 2), describe the Interaction Room method we introduced (Section 3), and report on the quantitative results (Section 4) and qualitative observations (Section 5) of our case study. After a discussion of these results (Section 6), we conclude with a discussion of related work (Section 7), a summary and an outlook on future work (Section 8).

2. Initial project situation

In our case study, we observed a team within a large-scale software project in which an inventory management system for health insurances is being developed. The system contains over 3000 user interface views covering a large variety of insurance cases. A number of project teams are simultaneously working on different components of this inventory system. The team we observed is a Scrum team of five developers on the development team, one Scrum master and one product owner. This team is both developing new modules for the system, and implementing change requests for existing modules. Each sub-project spans about three months. At the time of our research, the team was working on a component for the automated receipt, remodeling, storage and dispatching of insurance data from external partners of the insurance company ("Project A"). The individual features of this new component were set forth in backlog items

that served as a basis for the team's implementation work. From the backlog items, the team sliced individual tasks to plan each sprint.

In Scrum theory, all this slicing should take place in sprint planning meeting 2. Nonetheless, Schwaber predicts that only 60% of a sprint's tasks will be defined in this meeting, and assumes that 40% of the tasks will become apparent later during the sprint [2]. In the project we observed, even fewer – about 20–30% of the tasks – were added later during the sprint. However, we believe this is still an undesirably high number of late-identified tasks: If the team has a deep understanding of the backlog items, it should be able to decompose them into virtually all the tasks necessary to deliver the required functionality. However, if the team is unable to identify all tasks early, it is probably lacking information about the backlog items. In consequence, the team would be committing to backlog items it has not fully understood.

In the project we observed, the team was indeed trying to carve useful tasks out of backlog items largely by intuitive "gut feeling" rather than by a concrete methodical approach. As a consequence, more than a quarter of all tasks were not identified in the sprint planning meeting 2, but only became obvious later in the course of the sprint. This severely affected the team's ability to perform meaningful project controlling, i.e. to make reliable time and budget estimates and keep deadlines. We suspect that many Scrum teams are struggling with this problem when identifying concrete tasks from backlog items.

Before we introduced our approach, the team had performed the sprint planning meeting 2 in separate groups (which somewhat deviates from Scrum practice "by the book"): The six developers formed three groups with two developers each. Each pair took backlog items from the issue tracking software to identify tasks for them, and noted these tasks on cards. The tasks on the cards were not discussed further with the rest of the team, but just put on a bulletin board as a reminder. Thus, two thirds of the team members did not know any details about the tasks for completing a particular backlog item, as they had not worked on identifying them. Tasks were formulated as short notes such as "create database table for medical service", so only the developers who created the task knew exactly how to create the database table (e.g. which attributes, primary key, relations to other tables etc. it should have). For the other developers, most tasks needed clarification before they could work on them. In a few extreme cases towards the end of the sprint, none of the team members could remember what a certain task meant and how it was necessary for the completion of a particular backlog item. This indicates a clear lack of thorough understanding of the backlog items.

Another critical aspect was the expertise of the team members. The team we observed had a good distribution of knowledge and expertise among its members. However, there were certainly some areas where one team member had stronger knowledge and more experience than another (e.g. in user interface design). Since items were framed vertically [4, 5] (every item involved the data, logic and presentation layer) but the two developers who sliced a backlog item into tasks might not have expertise in all those layers, the task breakdown did not benefit from many developers' expertise, and the set of identified tasks remained incomplete.

The challenges that this incomplete task breakdown posed in practice can best be illustrated by looking at the burndown charts the team produced over a number of sprints. A burndown chart measures progress during a sprint by showing the remaining effort (in terms of person days or number of tasks) on the vertical axis, while the horizontal axis shows the time elapsed in the sprint [5]. Since the developers did not manage to identify all tasks at the beginning of the sprint, it became inevitable that additional tasks had to be added during the sprint to really complete the backlog items. That rendered the burndown charts less meaningful and therefore less valuable. This effect can be seen in Fig. 1, where the total number of known tasks (the rising red line at the top of both burndown charts) *increases* over the

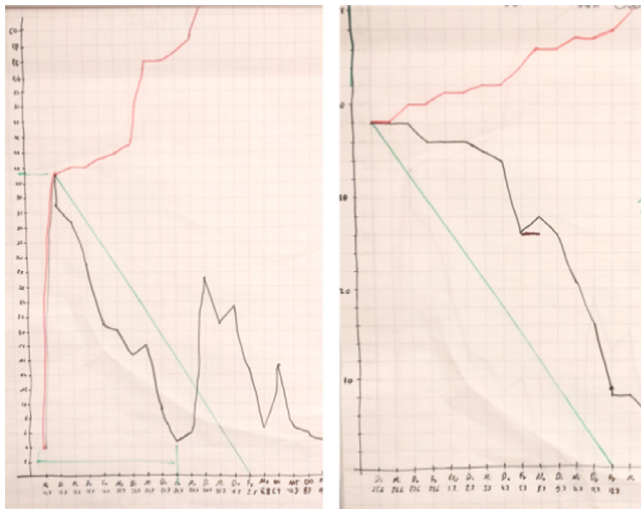


Fig. 1. Task burndown charts before adoption of the Interaction Room.

course of the sprint to exceed the number of initially identified tasks, instead of remaining constant. (Note that this does not constitute a violation of the requirements freeze that Scrum mandates during a sprint: The additional tasks do not represent commitments to new requirements (i.e. new backlog items), but represent previously unrecognized work that turned out necessary to complete the backlog items in the sprint backlog.)

If too many tasks are added subsequently during a sprint, it becomes much harder for the Scrum master (who is responsible for identifying and eliminating impediments to the team's work) to track progress based on the chart and to recognize and deal with project risks, as Fig. 1 illustrates. In this figure, the burndown charts of two sprints depict the situation before the introduction of our method. On the left is a burndown chart where 41 tasks were identified in sprint planning meeting 2, but another 16 tasks were identified over the course of the sprint (indicated by the rising red line at the top of the chart), so 28% of tasks were added during the sprint. In the sprint charted on the right, 38 tasks were initially identified in sprint planning meeting 2, but 12 more tasks were identified over the course of the sprint, so 24% of the tasks were added late. The total number of tasks increased steadily here (again shown in the red rising line at the top of the chart) as the team gained a deeper understanding of the backlog items and resulting tasks every day. On average, 26% of the tasks in this project's sprints were identified late in the sprints, instead of in sprint planning meetings already. This led to significant planning uncertainty in the company, as other components relied on functionality that could occasionally not be finished on time due to the additional discovered complexity (as shown by the jagged black lines in the middle of each chart, which tend to run considerably above the ideal burndown rate indicated by the monotonically falling green lines at the bottom of the charts).

3. Applied intervention: the Interaction Room

The above observations lead us to the conclusion that the team had difficulty with the derivation of tasks from backlog items. To improve this step, we suggested they adopt an approach that we believed would support the team members' understanding of the backlog items without introducing a heavyweight design step into the agile process model. In the following sections, we first introduce the intervention we suggested to the team in general, and then describe how it was applied in the context of the team's projects, before presenting the observed results in Sections 4 and 5.

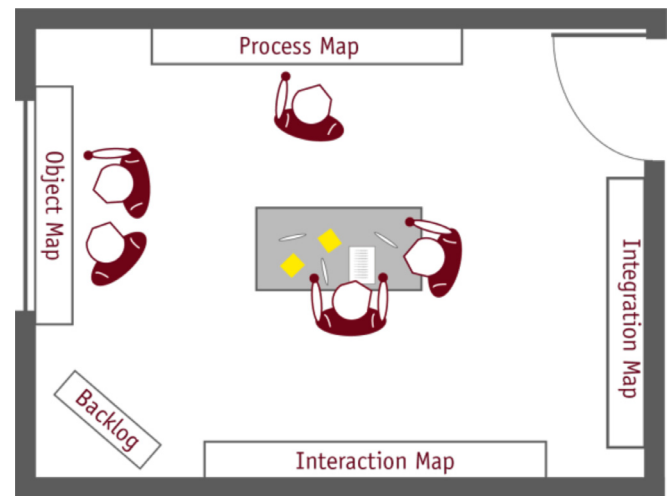


Fig. 2. Basic layout of an Interaction Room.

3.1. General Interaction Room approach

In order to provide the team with a more structured, yet pragmatic approach to derive tasks from backlog items, we encouraged the team to examine backlog items explicitly from four different perspectives that focus on static and dynamic, user- and system-focused aspects of each component and thus help the team to gain a comprehensive understanding of the system.

Firstly, since the main purpose of information systems is to support business processes, a backlog item should be described from a *business process* perspective. Within the processes, the information system works on business objects, so the second important perspective is the *object* perspective. Thirdly, an information system is usually highly integrated into an application landscape, and the system communicates with other systems through interfaces. Thus, the third perspective is an *integration* perspective. Finally, an information system is also a socio-technical system, so the team needs to consider backlog items from an *interaction* perspective. For each of these perspectives, we encouraged the team to examine backlog items and model the core process, data, integration and interaction aspects they entail. To facilitate this pragmatic, multi-perspective design sketching, we employed a so-called Interaction Room whose layout reflects the four perspectives [6]. This can be a dedicated conference room or (as in our case) an area of the team room equipped with large whiteboards. Every whiteboard is dedicated to the representation of one perspective, as shown in Fig. 2:

- The *backlog* is adopted from agile practice. It is used to manage the project's requirements (backlog items).
- The *process map* describes the sequence of business activities to execute the functionality described by a backlog item. It shows alternative and concurrent activity flows and relevant constraints.
- The *object map* describes the internal data structure of the system, showing data entities on a high level of abstraction. It also shows relationships between data entities. The object landscape remains on a business level, so implementation details, such as composition dependencies, are left out.
- The *integration map* shows the interfaces of the system under development to other systems. Incoming and outgoing interfaces show the exchanged data and possibly the communication protocols.
- The *interaction map* describes the user interaction on different levels of abstraction. A navigation overview highlights the menu structure and underlying dialog flows. Additionally, the graphical user interface can be sketched as a storyboard.

The model sketches in these maps remain deliberately abstract, informal and incomplete, as they are not intended to replace formal system specifications. Rather, they should serve as a catalyst for the discussion and joint understanding of a project and its possible pitfalls. We deliberately talk about “maps” instead of “models” to emphasize their main goal: empowering users to understand and discuss the crucial parts of a system, and use the model sketches as maps to orient themselves in the project. For this purpose, we also do not prescribe particular modeling languages for the various maps, but let teams choose a notation that all stakeholders understand. This is in accordance with Cohn, who states that there should be some kind of design in sprint planning meeting 2, but recalls that he has “never been in an iteration planning meeting where it’s become necessary to draw [a] class diagram or similar model” [5].

3.2. Application of the Interaction Room in the case study

In the projects we observed, the sprint planning meeting 2 was timeboxed between 1 and 4 h (depending on the sprint length). Thus, the team had limited time to make an item’s design explicit and to identify the tasks to implement and test it. We therefore discouraged team members from attempting to create an explicit, complete technical design model for each backlog item in this short timeframe, but rather recommended that they focus on understanding the subject matter through model sketches and getting a more solid impression of the technical steps required to implement any particular item.

This pragmatic focus on sketching those aspects of a system that are not yet sufficiently understood, without the ambition to create comprehensive specifications, fits well into the work philosophy of agile approaches and does not incur significant effort overhead – indeed, the (timeboxed) sprint planning meetings did not run longer after introduction of the Interaction Room than before, but the participants were more focused on understanding all aspects of the requirements. This was possible since the discussions in the Interaction Room were not intended to produce additional comprehensive specifications, but rather give the sprint planning meetings a structure and a visualization mechanism that allowed team members to explore the process, object, integration and interaction perspectives more explicitly.

To discuss these perspectives in sprint planning meeting 2, we let the whole Scrum team meet together, not just (as previously done)

the developers. This way, the product owner could work with the development team and provide feedback and information about the backlog items as required. The Scrum master moderated the meeting and encouraged the team members to participate in the discussion about the backlog items and the tasks (the Scrum master was chosen to moderate the meeting so the product owner would be on a level field with the developers). The developers discussed each backlog item based on its content: The team first decided if a backlog item would be triggered by a user or a system event. This decision was made by discussing the item between the development team members. For a user-triggered item, the team modeled the corresponding user interface views and sketched prototypes for them. For system-triggered items, the team modeled the event that triggers the business process. The Scrum master as the moderator was in charge of documenting the model sketches for later reference, but any team member was encouraged to actively join the design process.

Afterwards, the team decomposed the functionality of each item into its atomic functions. Once the atomic functions were identified, the team sketched a design for them by performing the following steps iteratively until the whole item was designed:

- Determine and model input data of the function
- Design function from a business process perspective
- Determine and model output data of the function

In case the input data would be provided by the user, the team modeled the user interaction and designed the data structures to store the provided data. On the other hand, if the input data would be provided by an outside system or component, the team designed the respective component interface. Since the output data could be stored as part of the internal data warehouse or written to a component or user interface, the team also sketched the respective data structures or interfaces in the Interaction Room.

The team repeated these steps of sketching designs from the data, process, integration and interaction perspective – not until they arrived at a complete specification, which would contradict the agile approach, but until all team members had gathered the necessary knowledge from the coarse sketches to understand the required functionality and its dependencies across the perspectives.

Fig. 3 depicts the coarse methodical guideline the team followed to examine backlog items and derive tasks from them. We believe that the task breakdown process cannot be described more formally,

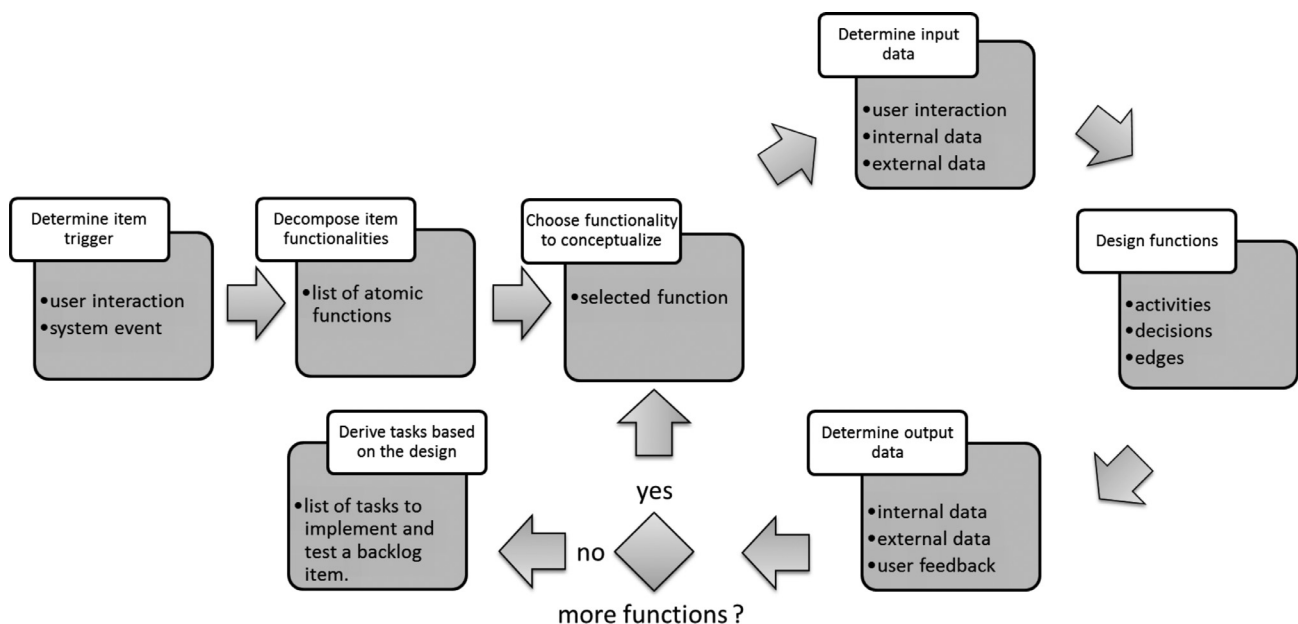


Fig. 3. Coarse guideline followed by the team to examine backlog items from different perspectives in order to identify tasks.

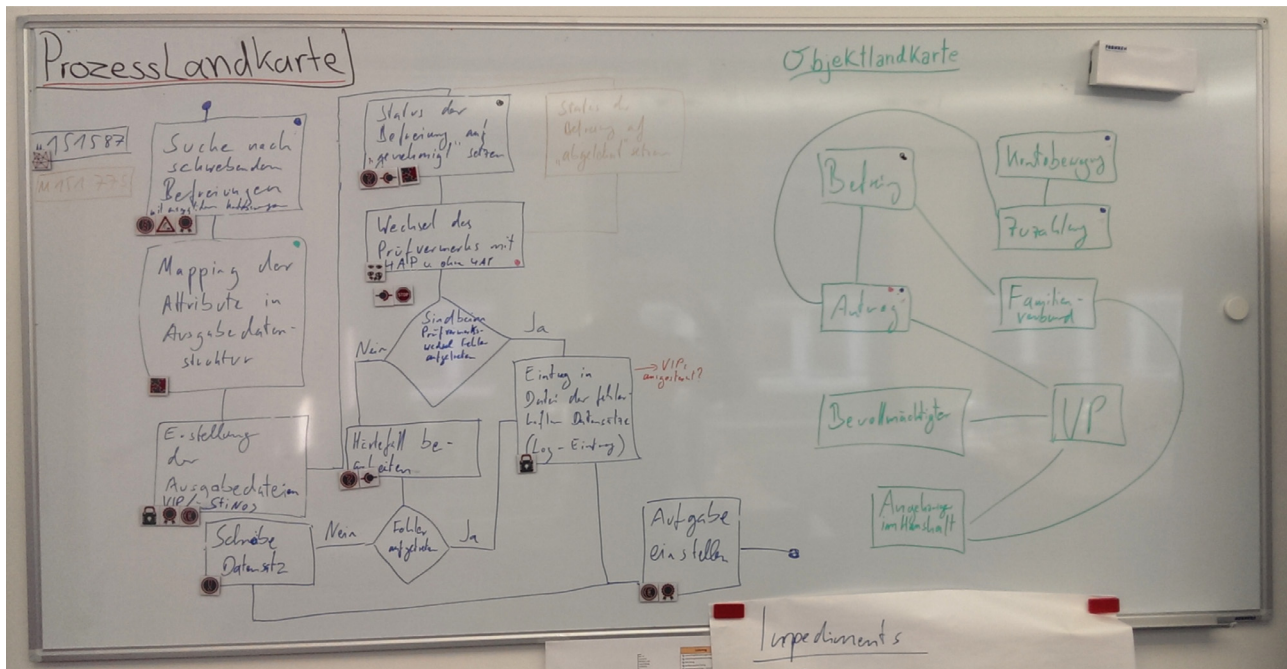


Fig. 4. Example of a process and object map in the Interaction Room.

and may vary somewhat depending on the nature of the project or application domain (mobile applications or embedded systems, for example, would likely require focusing on some similar, but also some different aspects). However, the core of the approach – encouraging collaboration among all team members, considering all business and technical perspectives, focusing on resolving uncertainty, but forgoing the ambition to produce a comprehensive specification – should help to raise issues that could be overlooked if the design was made in small groups, or if there was no explicit consideration of the various design aspects in the sprint planning meeting at all.

3.3. Interaction Room usage example

To provide an impression of the level of abstraction pursued in the Interaction Room, Fig. 4 shows an example of a process map and object map that were sketched by the team. The process map describes an automated process for notifying customers of public health insurance companies.

The level of abstraction in this top-level sketch was quite high, as indicated by labels such as “searching for data”, “mapping of attributes to output structure”, “creation of output files”, “change of status” etc. After sketching the process map, the team went on to sketch the corresponding object map. They went through the individual process steps and determined which data needed to be processed in each particular step. In sketching the object map, the team focused on the business objects that are processed in the software module, e.g. “bank account”, “application”, “insured person” etc. The team paid special attention to designing the maps in such a way that each entity shown on the object map was manipulated by some activity on the process map, and each entity known to be manipulated in the course of an activity on the process map was also noted on the object map.

With the understanding and knowledge gained from this modeling process, the team was convinced they knew enough about the component to be built in order to understand and estimate some of the backlog items. The team then went into greater detail and modeled the missing process steps on the process map, added missing objects to the object map and designed the user interface on the interaction map (not shown here for brevity). The integration map reflecting

the technical implications of the process in this case comprised only one call to a particular service in a different software component.

After this, creating the individual tasks that were necessary to fulfill the acceptance criteria of a backlog item appeared relatively easy, compared to the difficulties the team had had while defining all the right tasks earlier. Because the whole business process, its data structures, user interfaces and integration implications were now understood well, the necessary tasks were clear to everyone and just had to be documented on cards which were added directly to the model sketches, as shown in Fig. 5. This provided all team members with an intuitive impression not just of a process step’s complexity, but also of the business interdependencies between the tasks. While the figure looks somewhat messy, this informal sketch contains just the level of detail that the team needed to understand the business process and identify the resulting tasks: The team members felt they could approach the design and implementation of each task with better consideration of the “big picture” this way, than when they just perceived tasks as isolated cards in a list of backlog items.

3.4. Research method

We applied the Interaction Room approach described above in two separate sub-projects (“Project A” and “Project B”) within the context of a large-scale project developing a health insurance management system governing the key business processes of a health insurance company (patient and healthcare provider inventory, premium payment and reimbursement, cost controlling etc.). The overall project was marked by considerable complexity due to the multitude of corporate and governmental stakeholders in the healthcare domain, as well as the large volume of legal requirements that had to be respected.

In Project A, the team was working on a component for the automated receipt, remodeling, storage and dispatching of insurance data from external partners. This project was already underway before we introduced the Interaction Room, and exhibited the initial task identification deficits described in Section 2. Following the action research paradigm, one of the authors, who was part of the project team, introduced the new approach to the other team members, who adopted it quickly. The burndown charts were recorded by the team over the

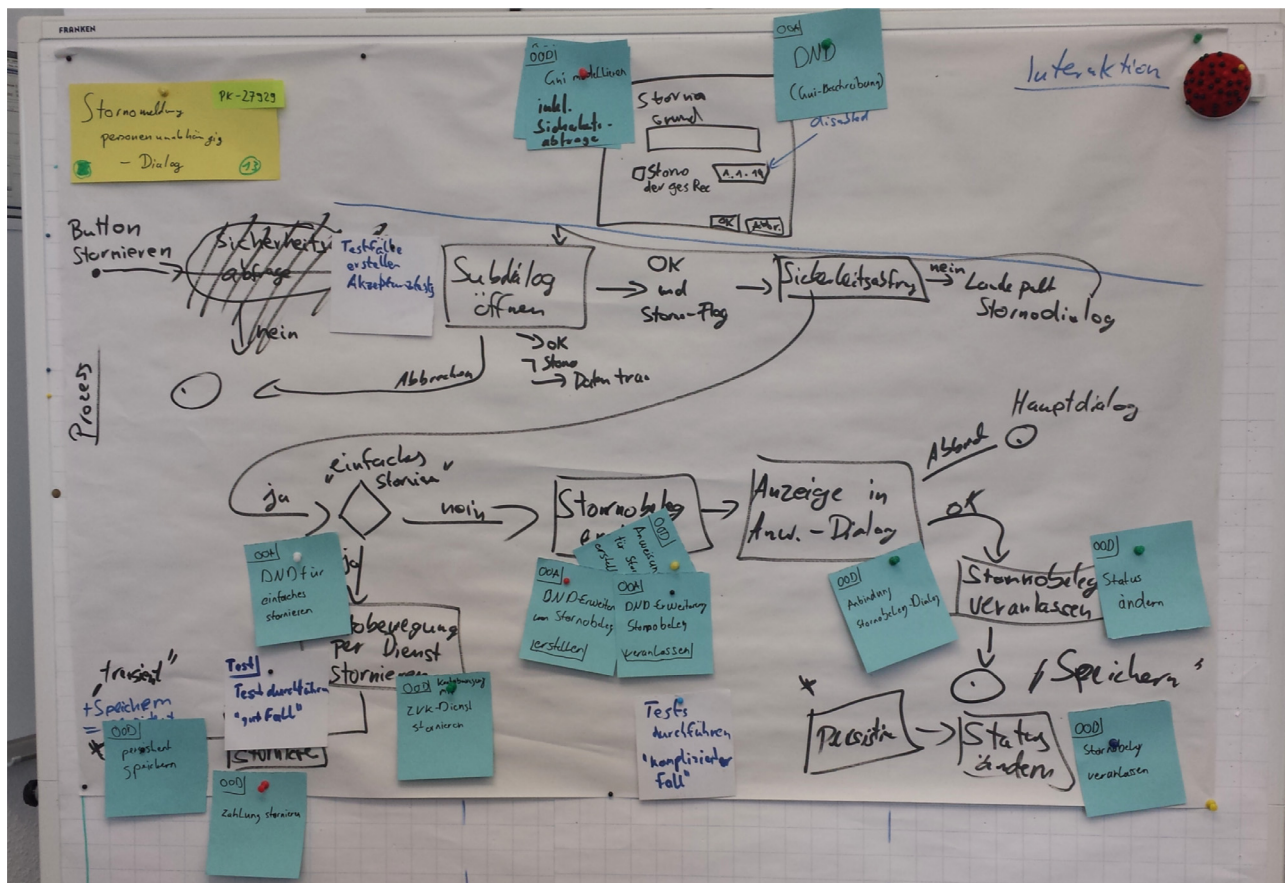


Fig. 5. Example of a task breakdown on the Interaction Room's process map, with tasks affixed directly to the process steps they refer to.

course of each sprint, based on day-to-day tracking of the identified and completed tasks. (We have redrawn the burndown charts in this article to improve readability. Some of the original hand-drawn charts can be found in [8].)

To examine if our findings were specific to the circumstances of Project A, we observed the same team working on three more sprints in a different project of comparable complexity ("Project B"), which focused on a component for reimbursing doctors for non-patient-specific expenses. In these sprints, the team also used the Interaction Room to facilitate the task breakdown in the sprint planning meetings. By the time Project B began, the team had gained considerable experience in using the Interaction Room and did not need any further methodical guidance. Since the team was working on a different part of the system than in Project A, we can also rule out that accumulated knowledge about the business requirements had an impact on the number of late-identified tasks.

The two projects A and B were chosen by virtue of being accessible to us, as one of the authors was part of the Scrum team on both projects. In both projects, the team was responsible for developing new modules for the health insurance system, and implementing change requests for existing modules. The data we analyzed in the study was gained from sprints where the overall project ran smoothly and without any of the organizational disruptions (such as changing management, procedures and deadlines) that cannot be avoided in a large-scale industry project, but would have impacted the composition of the backlog and the execution of the sprints, and thus skewed the counts of early- vs. late-identified tasks.

To test our hypothesis that the more intensive and focused communication in the Interaction Room helps to achieve a more comprehensive task breakdown, we calculated the percentage of late-identified tasks in the post-intervention sprints and compared it

to the percentage of late-identified tasks in the pre-intervention sprints.

In addition to our previously published findings from Project A [8], this article provides additional quantitative and qualitative observations from Project B. We present the quantitative results from both projects in the following section, report on our qualitative observations in Section 5, and discuss the overall results and limitations of the case study in Section 6.

4. Quantitative results

4.1. Task identification in Project A

After introducing the Interaction Room to the Scrum team, we observed their performance over the course of three sprints in Project A to assess the impact of our approach. As an indicator for its benefits, we tracked the tasks that were added late during each sprint. In these three sprints, 19 backlog items were implemented. These items were broken down into 155 tasks, of which 145 tasks were derived in sprint planning 2 meetings and 10 tasks were added subsequently. Thus, in total only 6.5% of the tasks were added late during a sprint. Fig. 6 shows the burndown charts of the three observed sprints.

As can be seen in these charts, the top lines (i.e. the number of identified tasks) remain relatively level for most of the sprint duration, and rise only slightly on occasion (in contrast to the strongly rising top lines in the charts in Fig. 1). In the first sprint, the team committed to implement six backlog items. In sprint planning meeting 2, 48 tasks were identified for these, and seven tasks were added during the sprint, i.e. about 13% of tasks were added subsequently. In the second sprint, seven backlog items were implemented, for which 53 tasks were derived during sprint planning meeting 2.

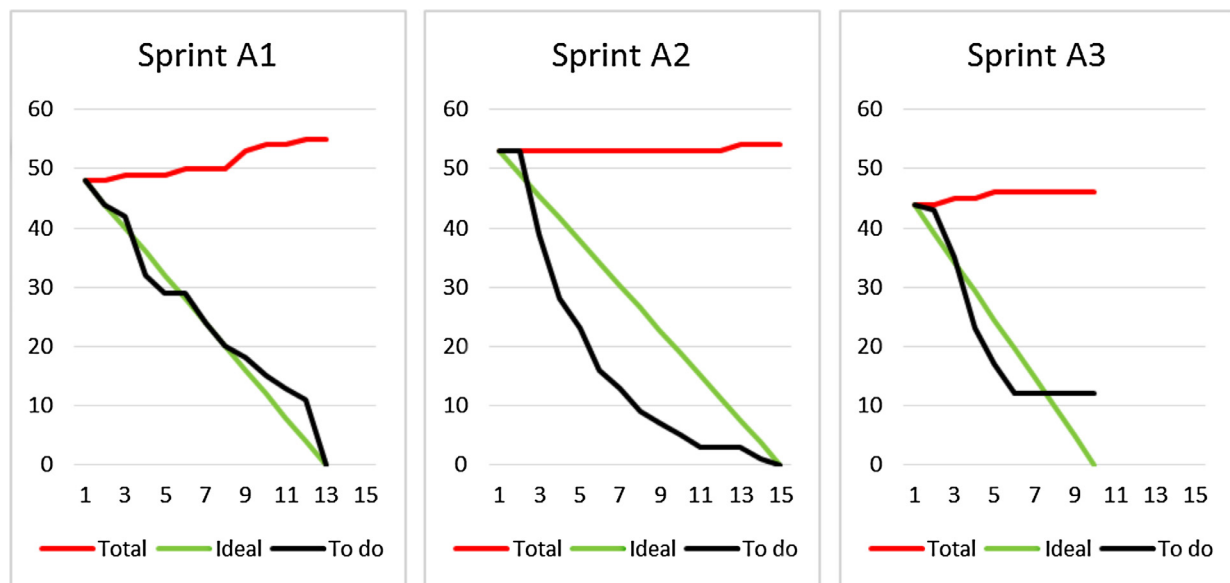


Fig. 6. Task burndown charts of sprints in Project A after adoption of the Interaction Room.

Table 1
Task breakdown results after adoption of Interaction Room in Project A.

Sprint	Tasks identified in initial meeting	Tasks identified later during sprint	Percentage of late-identified tasks
A1	48	7	12.7%
A2	53	1	1.8%
A3	44	2	4.3%

Table 2
Task breakdown results using the Interaction Room in Project B.

Sprint	Tasks identified in initial meeting	Tasks identified later during sprint	Percentage of late-identified tasks
B1	42	2	4.5%
B2	54	2	3.6%
B3	52	2	3.7%

Just one task was added later during the sprint, i.e. about 2% of the tasks were added subsequently. In the third sprint (which was timeboxed with just two weeks to make time for a monthly “integration week”), six backlog items were committed to, for which 44 tasks were derived during sprint planning meeting 2. Two additional tasks were added during the sprint, i.e. about 4% of the tasks were added late here (Table 1).

In total, our observations of the sprints before and after the introduction of the Interaction Room in Project A show that the percentage of late-identified tasks fell from the previous average of 26% to less than 6.5% on average after the new approach had been introduced.

4.2. Task identification in Project B

In the sprints of Project B, the number of late-identified tasks remained low, as Table 2 shows: In sprint B1, three backlog items were completed. Those three backlog items were split into 44 tasks, two of which were identified during the course of the sprint. This means less than 5% of the tasks were identified late in that sprint. In sprint B2, the team completed seven backlog items that were broken down into 56 tasks. There were two late-identified tasks among those, which yields a late-discovery rate of less than 4%. In sprint B3, the team completed 10 backlog items and broke them into a total of 54 tasks, two of which were identified late during the sprint – a late-identification rate of

less than 4% as well. Overall, the team worked on 20 backlog items in Project B. In order to complete them, the team identified 154 tasks in total, six of which were identified late during the sprints. This means that only 3.9% of the tasks were identified late across all sprints.

Fig. 7 shows the burndown charts of the three observed sprints in Project B. The task plots (i.e. the top lines) in these charts mostly rise only marginally, showing that the majority of tasks was identified in the initial sprint planning meeting 2, which enabled a relatively smooth task burndown.

Two unusual turns in these task plots beg an explanation: In sprint B1, a backlog item that involved seven tasks was cancelled by the customer at the end of the sprint, leading to the final drop in the total task plot. In sprint B3, the team had managed to complete all planned tasks in their backlog already two-thirds into the sprint, so they added two further backlog items to the sprint. The two jumps in the total task plot reflect the additional tasks, which were again identified using small Interaction Room workshops. Since neither the changes in B1 nor in B3 were related to task identification, but rather due to strategic decisions about backlog item inclusion, we have disregarded them in Table 2.

5. Qualitative observations

5.1. Observations in Project A

In our observations of Project A, two side benefits of the approach became apparent: Firstly, since the team identified almost all tasks already in sprint planning meeting 2, it was possible to measure the sprint velocity much more reliably, since the number of tasks in the burndown chart remained largely stable, so the team members had a more realistic impression of the progress they actually made. Secondly, when using the Interaction Room, the whole team collaborated on the design sketches and the task decomposition, so every team member became able to understand or (with less extra study) even perform every task, whereas the relevant knowledge about individual tasks had been much more compartmentalized before.

In addition to the quantitative results, we also asked the team members to give us their informal opinion about the new approach. They responded that previously, they had sometimes felt uncertain why and how to perform a task during a sprint, but now felt that the new collaborative design fostered joint understanding about the backlog items, derived tasks and their rationales. This was particularly important in this project, since every backlog item was

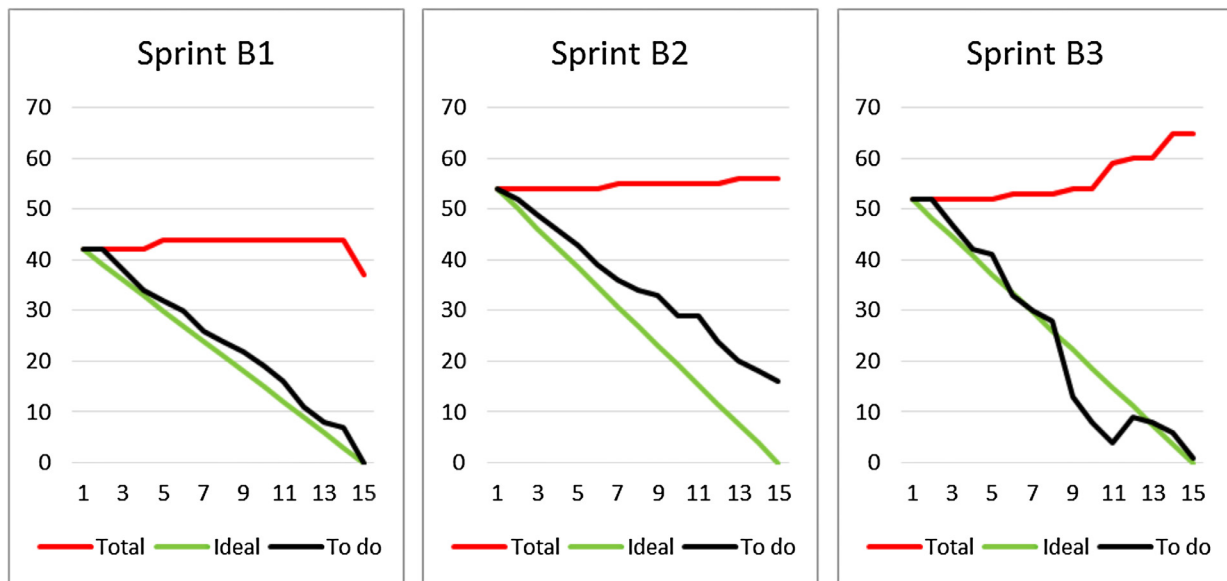


Fig. 7. Task burndown charts of sprints in Project B using the Interaction Room.

designed separately instead of in context of the whole system (and thus other backlog items). We conjecture that we could achieve further improvements in the completeness of the identified tasks if the relationships between various backlog items were taken into account, as these can be quite easily visualized in the Interaction Room.

5.2. Observations in Project B

In Project B, we had the opportunity to observe more closely how the team conducted their sprint planning. We established a dedicated observer in each sprint planning meeting 2, who e.g. noted which aspects were discussed in the meeting, and which ones were modeled on the Interaction Room walls. We also examined if the reduction in late-identified tasks could be attributed to other aspects of the team's conduct in the task breakdown meetings. Two effects stood out in this analysis, which – although anecdotal – shed additional light on how an Interaction Room seems to support understanding of the tasks underlying a backlog item.

5.2.1. Distinct treatment of development and testing tasks

We found that the comprehensiveness of the identified tasks depended not just on the task breakdown method, but also on the involved roles and the communication between them: While we had encouraged the whole development team to work together on the task breakdown in Project A, our observation of the team's conduct in Project B suggests that this is not always necessary. In Project B, the development team had an unusual component: One of the team members was not involved in writing code, but instead responsible for ensuring the correctness of the implemented business processes. One part of his role was to support the product owner in creating the backlog items. Another part was to serve as a source of expert knowledge on health insurance processes for the other members of the development team.

From a pure Scrum perspective, this is unusual since a Scrum team should strive for homogeneity, with similar levels of knowledge and skill shared among team members. It could be argued that the above-mentioned responsibilities of this domain expert should lie outside the development team – in fact, maybe even outside the Scrum team. After all, in Scrum, the product owner should be solely responsible for defining and prioritizing product requirements, and thus be responsible for the outcome of the product – the “single wringable neck” [3]. However, it is not forbidden for the product owner to seek support from domain experts.

The team member in question had an additional role though: He was also the quality assurance (QA) expert on the development team, so his responsibility also included defining the test cases for the backlog items and testing the software increments before they were presented to the product owner for final approval. In our observations of Project B, we noted that the QA expert often worked on the identification of the test tasks by himself in sprint planning meeting 2, while remaining in close contact with the product owner and presenting the test tasks to the rest of the development team regularly. Occasionally, this led the members of the development team to identify further development tasks after learning about the tests that would be conducted for a particular backlog item. Conversely, however, the relatively isolated identification of test tasks at the beginning of the sprint did not adversely affect the number of late-identified test tasks.

This seems to indicate that it is not necessary for the QA expert to know about the development tasks before identifying the test cases, while it *does* seem valuable for the development team to know the test tasks when identifying the development tasks. When using the Interaction Room method, it therefore seems to be not strictly necessary to involve the QA expert in identifying development tasks. This way, the task breakdown of development and test tasks can be somewhat parallelized and thus sped up.

In our experience, it is common in practice that development teams are not completely homogeneous. Expertise, especially with regard to quality assurance, is usually not distributed evenly among the development team members. It is not uncommon to have one team member who is mainly responsible for test planning. Often, the same person is responsible for writing the acceptance tests for the backlog items, and possibly even for the bulk of testing activities. Therefore, we believe that the above insights into the need for this person's inclusion in the development task breakdown will be applicable to other teams as well.

5.2.2. Choice of abstraction level during task breakdown

When modeling the different perspectives on a project in an Interaction Room, it is often a question which level of abstraction would be appropriate for understanding a particular backlog item: If the level of abstraction is too detailed, the team would spend more effort than necessary in this early phase (which should just be focusing on task breakdown), and may be tempted to sketch ad hoc solutions to issues that would merit more in-depth analysis. On the other hand, if

the model sketches in the Interaction Room remain too high-level, the team may get a false sense of simplicity and thus a too shallow depth of understanding, provoking the late discovery of additional tasks since the implementation challenges were not properly understood at the time of task breakdown.

While observing the Scrum team in Project B, we noticed an interesting effect that may suggest a reasonable level of abstraction that should be aimed for in a sprint planning meeting 2: One backlog item included a process step called “assemble order information”. The team members briefly spoke about which data needs to be assembled and what needs to be validated, and then agreed that no further examination of that process step was necessary since the validation just consisted of “some simple checks”. The process step “assemble order data” was therefore not modeled in the Interaction Room, and the task derived from that backlog item was also simply called “implement assembly of order data”, with an estimated effort of one person day. During the sprint, this single task ended up requiring more than 10 person days to implement, and four defects were later logged for this backlog item.

This illustrates that a too high level of abstraction can lead to defects and severe misestimates. Our recommendation therefore is that any backlog item should be refined at least to the degree that it is not represented by a single “black box” in the Interaction Room, to ensure a reasonable attempt is made at understanding its internal structure and uncovering possible hidden challenges.

6. Discussion

6.1. Conclusions from the case study

Our observations of the sprints before and after the introduction of the Interaction Room in Projects A and B show that the percentage of late-identified tasks fell from the previous average of 26% to about 5% after the new approach had been introduced – much less than the 40% late-identified tasks that even Schwaber considered inevitable [2]. We hypothesize that this is due to the more comprehensive examination of each backlog item from the four perspectives of the Interaction Room, and in particular the discussion sparked among the team members while considering and modeling a backlog item’s structural and functional implications in the overall system.

Our case study was not designed to identify the precise mechanisms that helped to achieve the improved task breakdown. The Interaction Room method encourages collaboration among all stakeholders; explicit consideration of process, data, integration and interaction aspects; identification of risk, effort and uncertainty drivers; informal visualization of those aspects; and a pragmatic focus on resolving open design questions rather than producing complete, detailed specifications. To which degree each of these mechanisms contributed to the overall outcome, which of them could possibly be forgone and which should be emphasized, would require a more formal experimental setup that enables the isolated measurement of each aspect. We are considering feasible setups for such experiments in our ongoing research.

Still, we are reasonably confident that the involvement of all stakeholders in the Interaction Room helped to arrive at a more complete picture of the work to be done. One might argue that the team’s original approach of splitting up the developers into pairs for the task breakdown was flawed to begin with – however, this was motivated by the idea that the task breakdown work could be parallelized by letting developers focus on those user stories they would later like to implement. Even if developers later ended up implementing other tasks than those they planned themselves (which is certainly not ideal), it can be argued that the people who performed the original task breakdown should have been more diligent in identifying all necessary tasks. Instead, we would consider as a more severe flaw the common Scrum practice of letting developers perform the task

breakdown by themselves, without necessarily involving all the other team members. This bars domain experts’ knowledge from the breakdown process, and thus leaves developers unaware of the complexity of the user stories they are supposed to break down. The Interaction Room reintroduces this expert knowledge into the breakdown process, and explicitly focuses communication on the complex and uncertain aspects.

6.2. Threats to validity

Given that our case study was undertaken within the confines of an actual large-scale industry project, we only had limited influence on controlling the conditions under which the intervention and data analysis took place. Still, we consider the construct validity of the study to be sound since our intervention (using the Interaction Room for task breakdown) follows a clearly defined methodology. While the dependent variable (comprehensive understanding of the subject matter and necessary development tasks) seems hard to capture at first sight, we believe that the number of late-identified tasks is an ideal metric since any such task is a clear indicator of a lack of early comprehensive understanding. Also, the number of late-identified tasks is easily and objectively measurable.

The data we analyzed in the study was gained from sprints where the project ran smoothly and without organizational disruptions such as deadline and priority changes that could have impacted the composition of the backlog and the execution of the sprints. Although we would have preferred to examine a project that ran smoothly for extended periods of time, and gave us control of as many variables as possible, we believe such projects are hard to find outside more artificial academic settings such as classroom projects.

Typical confounding effects that might play a role in other experimental settings (such as observer, practice or carryover effects) can however be excluded here: Since the continuous tallying of progress metrics (including the number of tasks) is a routine activity in agile project management, our data collection did not convey any unusual expectancy. In particular, the team members were not aware that we would evaluate the number of late-identified tasks after introduction of the new method. Also, since the person who introduced the Interaction Room method to the team was a regular team member and not an external consultant, an observer effect can be discounted.

The application of the method to two projects of course means that the team was more familiar with the method in the second project. However, the team was initially just as unfamiliar with the subject matter of Project B as it had been with Project A’s, so the dependent variable (understanding of the subject matter, in terms of late-identified tasks) was not biased by learning due repeated application of the method.

With regard to internal validity, we cannot claim with absolute certainty that the observed improvement in task breakdown efficiency was caused solely by using the Interaction Room – two other potentially contributing factors could be the team members’ increasing familiarity with the subject matter, as well as varying complexity of the sprints’ tasks. However, the general level of task complexity was quite homogeneously high across the various sprints, and the sharp drop to just one-fifth of the previous number of late-identified tasks seems inconsistent with a gradual familiarization process. Also, the switch to the Project B, which had a different subject matter, rendered a lot of knowledge gained on the subject matter of Project A obsolete, eliminating the potential of learning effects. The fact that Project B, which employed the Interaction Room from the start, exhibited the same low number of late-identified tasks right from the start also indicates that a learning effect did not play a role.

Other confounding factors could be suspected in the team’s composition, in particular the role of the person who introduced the Interaction Room method to the team. Since the team composition remained unchanged through both projects, we can discount

particular members' competencies or characters as relevant influences on the observed number of late-identified tasks. Also, while the team member who introduced the Interaction Room did provide continuous methodical guidance on how to use the approach, he had no deeper insight into the subject matter than the other team members had, and thus could not support the team's understanding of the subject matter in independent ways that would confound the Interaction Room's impact.

We must point out that our case study was not designed to identify which individual aspects of the Interaction Room method (e.g. the consideration of distinct process, object and integration perspectives, the acceptance of uncertainty and imprecision, or the presence of a team member who continuously reminded the team members of this big-picture view) are most effective in obtaining the desired better understanding of the subject matter. The study does however allow us to examine the impact of the Interaction Room method as a whole.

Regarding the external validity of our results, we must bear in mind that these results are non-representative since they stem from a particular project of a particular team in a particular domain. Before we can make claims of wider generalizability, the approach therefore also needs to be evaluated in other project settings (e.g. smaller projects, different domains, schedules, team compositions etc.). However, we consider this case study instructive nevertheless since it shows how a significant improvement in task breakdown effectiveness can be accomplished even under the quite challenging conditions of a complex, real-life industry project.

7. Related work

The smooth execution of agile projects hinges to a considerable degree on the stakeholders' ability to partition the overall work into sprints and tasks of manageable size. A number of effort estimation techniques have been proposed for this purpose over time, albeit with limited accuracy [9]. Beside reliable estimation techniques (which are not the focus of our work), our experience from working on the health insurance management system showed us that understanding the subject matter is the necessary foundation for realistic sprint planning – any estimation technique can only be as good as the stakeholders' understanding of the subject matter. As our pre-intervention project experience (Section 2) showed, understanding a project's subject matter at just the right level of detail to define a realistic sprint backlog, but not get bogged down in details, is not trivial, and can severely impact a team's ability to deliver features reliably.

With the Interaction Room, we therefore introduced a method that strives to focus team communication more effectively. The importance of physical artifacts for supporting communication and co-ordination was investigated before (see e.g. [10, 11]). The idea of using dedicated physical rooms for system development bringing users with different backgrounds and viewpoints together can be traced back to the Joint Application Design (JAD) approach that was developed by IBM in the late 1970s [12]. We adapted this approach by focusing the Interaction Room method on pragmatic modeling and cross-domain discussion, in order to achieve a broader understanding and resolution of uncertainties in all business and technical aspects of a software system. Comparable contemporary approaches can e.g. be found in the visual architecture modeling techniques of Mangano and van der Hoek [13].

The importance of human and social factors in software teams in general, and of process models that support team communication in particular, has been the subject of numerous studies as well, and agile approaches have been shown to be particularly effective in fostering increased team communication. Sharp and Robinson, for example, examine relationships between XP and organizational culture [14], and show that technical practices have an impact on social interactions among team members [15]. Cristal et al. [16] and Moore

et al. [17] also found improvements in communication through the introduction of Scrum.

The foundation of the productivity improvements reported for agile practices, however, is not improved *communication*, but improved *cognition*. Preece et al. [18] describe the following aspects that need to be considered in a cognitive system with multiple collaborators:

- The mechanism of distributed problem-solving.
- What is said and what is not said, but implied and equally important.
- The kind of co-ordination that is facilitated by humans.
- The communication paths among collaborators.
- The mechanisms of knowledge sharing, and the way knowledge is accessed.

In our case study, these aspects contributed in several ways: We observed distributed problem-solving already in the pre-intervention phase, when different team members worked on the task breakdown of different user stories. This distribution however created more conflict than synergy, as the team members were not aware of each other's solutions, but focused only on their own slice of the work. In the Interaction Room, in contrast, the team members worked on understanding all user stories together. Distributed problem solving occurred here in two ways: First, by looking at the process, data and integration aspects of the implementation separately, the team members were encouraged to tackle the same problems from different perspectives. Secondly, each team member could address the problem based on his or her personal expertise (e.g. in business, implementation, testing or operational matters), and the individual contributions were combined into an overall solution.

This approach likely also affected the amount of important knowledge that was not voiced, but only implied: Addressing challenges from different viewpoints, and in particular encouraging the articulation of uncertainty, could have reduced the barriers to knowledge sharing that can otherwise be present in more homogeneous teams or teams dominated by technical stakeholders. In our case study, the team frequently encountered situations during the pre-intervention sprints where team members did not share key factual knowledge because they thought it would be obvious, or because they were uncertain.

With regard to communication paths and coordination mechanisms, one might assume that agile methods already provide sufficiently explicit guidelines. In practice, however, these guidelines prescribe only the *format* of the communication (who, when, how long), but do not guarantee its *substance*. The Interaction Room method fills this gap by focusing discussion on those aspects of the system that are of particular value, that are particularly risky, and that are subject to uncertainty – i.e. those aspects that are most critical for the success of the project.

Finally, the Interaction Room method facilitates knowledge sharing because it preserves the results of the task breakdown discussions in visual form on the team room's whiteboards, where they can be accessed and amended at any time during the sprint. In contrast to traditional specification documents, the creation of these sketches does not require extra effort though, because they are a natural by-product of the task breakdown meeting. Also, we found that the Interaction Room's syntactical leniency encourages stakeholders with little or no formal modeling training to share and visualize their knowledge. Stakeholders' preference for physical rather than electronic means of collaboration (as reported by Whittaker and Schwarz [11]) may play an additional role in the effectiveness of the approach.

Looking at the process of collaborative modeling in particular, Renger et al. identified four trade-offs [19] that are also addressed in our approach:

- Group composition: We explicitly encourage mixed groups including both business and technical stakeholders. The potential

misunderstandings mentioned by Renger et al. are far outweighed by the gain in diversity of perspectives, in our experience, and a dedicated moderator can help to defuse any potential conflicts.

- Level of participation: While Renger et al. see benefits in empowering stakeholders to make model changes themselves, they are wary of the modeling skills it takes to produce syntactically correct models. In the Interaction Room, we also encourage active modeling by the stakeholders, and deliberately relax the syntactic quality requirements in order to make the entrance barrier low for everyone. Given that we are not interested in precise system specifications, but in a general understanding of the domain requirements at this point, we have not found this initial syntactic leniency to be problematic.
- Starting point: Regarding the question of whether to start modeling on a clean slate, or prepare some preliminary models for stakeholders to start with, we preferred not to prepare preliminary models since the overall health insurance project typically provided enough background knowledge that could serve as a frame of reference for stakeholders.
- Parallel modeling: While there was a certain degree of distributed problem solving, as discussed above, we deliberately did not parallelize the actual modeling process as this is the activity where the different stakeholders' perspectives need to converge, and where consensus must be reached on key business and design decisions. As Renger et al. argue, parallel modeling would indeed preclude this convergence.

Using the Interaction Room thus provided one approach to dealing with the various collaboration challenges described in the agile software development literature. Given the specificity of our project context, further research is necessary though to examine the practicality and benefits of the approach in general practice.

8. Conclusions

Our observations of two large Scrum-based industry projects showed that about a quarter of all tasks (26%) were originally not identified in sprint planning meeting 2, but only became apparent later during the course of the sprint, indicating that the team members were having difficulty deriving all necessary tasks from a sprint's backlog items without any methodical support. We therefore encouraged team members to analyze backlog items and their resulting tasks in an Interaction Room that facilitated pragmatic sketching and discussion of designs to obtain a high-level overview of each backlog item. Using this approach, the team could identify the tasks necessary to implement and test individual backlog items more confidently and comprehensively – in two independent projects, just about 5% of tasks were now added late during a sprint on average.

In our ongoing research, we are continuing to observe other agile projects to obtain further data on the impact of our approach on task breakdown, generalize our findings and identify the methodical elements that are primarily responsible for the observed improvements.

Furthermore, we will also investigate if there could be a complexity measure for backlog items that suggests when a more in-depth analysis in the Interaction Room is advisable. Another interesting field of study that is suggested by the findings in this article is which factors in sprint planning have an effect not just on the number of identified tasks, but also on the number of defects that accumulate during a sprint.

References

- [1] C. Schwaber, G. Leganza, D. D'Silva, The Truth About Agile Processes, Forrester Research, 2007 <https://www.forrester.com/The+Truth+About+Agile+Processes/fulltext/-/E-RES41836>.
- [2] K. Schwaber, *Agile Project Management with Scrum*, Microsoft Press, 2004.
- [3] K.S. Rubin, *Essential Scrum. A Practical Guide to the Most Popular Agile Process*, Addison-Wesley, 2012.
- [4] M. Cohn, *User Stories Applied: For Agile Software Development*, Addison-Wesley, 2004.
- [5] M. Cohn, *Agile Estimating and Planning*, Prentice Hall, 2005.
- [6] M. Book, S. Grapenthin, V. Gruhn, Seeing the forest and the trees: focusing team interaction on value and effort drivers, in: Proceedings of ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering (FSE), New Ideas Track, ACM, 2012, pp. 30:1–30:4, doi:10.1145/2393596.2393630.
- [7] S. Grapenthin, M. Book, V. Gruhn, C. Schneider, K. Völker, Reducing complexity using an interaction room: an experience report, in: Proceedings of 31st ACM International Conference on Design of Communication (SIGDOC), ACM, 2013, pp. 71–76, doi:10.1145/2507065.2507087.
- [8] S. Grapenthin, S. Poggel, M. Book, V. Gruhn, Facilitating task breakdown in sprint planning meeting 2 with an interaction room: an experience report, in: Proceedings of 40th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), Software Process and Product Improvement (SPPI) Track, IEEE, 2014, pp. 1–8, doi:10.1109/SEAA.2014.71.
- [9] M. Usman, E. Mendes, F. Weidt, R. Britto, Effort estimation in agile software development: a systematic literature review, in: Proceedings of 10th International Conference on Predictive Models in Software Engineering (PROMISE), IEEE, 2014, pp. 82–91, doi:10.1145/2639490.2639503.
- [10] H. Sharp, H. Robinson, Collaboration and co-ordination in mature eXtreme programming teams, *Int. J. Hum.-Comput. Stud.* 66 (7) (2008) 506–518, doi:10.1016/j.ijhcs.2007.10.004.
- [11] S. Whittaker, H. Schwarz, Meetings of the board: the impact of scheduling medium on long term group coordination in software development, *Comput. Supported Coop. Work* 8 (3) (1999) 175–205, doi:10.1023/A:1008603001894.
- [12] E. Carmel, R.D. Whitaker, J.F. George, PD and joint application design: a transatlantic comparison, *Commun. ACM* 36 (6) (1993) 40–48, doi:10.1145/153571.163265.
- [13] N. Mangano, A. van der Hoek, The design and evaluation of a tool to support software designers at the whiteboard, *Autom. Softw. Eng.* 19 (4) (2012) 381–421, doi:10.1007/s10515-012-0104-9.
- [14] H. Robinson, H. Sharp, Organisational culture and XP: three case studies, in: Proceedings of Agile Conference, IEEE, 2005, pp. 49–58, doi:10.1109/ADC.2005.36.
- [15] H. Robinson, H. Sharp, The social side of technical practices, in: Proceedings of 6th International Conference on Extreme Programming and Agile Processes in Software Engineering (XP), LNCS 3556, Springer, 2005, pp. 100–108, doi:10.1007/11499053_12.
- [16] M. Cristal, D. Wildt, R. Prikladnicki, Usage of Scrum practices within a global company, in: Proceedings of IEEE International Conference on Global Software Engineering (ICGSE), IEEE, 2008, pp. 222–226, doi:10.1109/ICGSE.2008.34.
- [17] R. Moore, K. Reff, J. Graham, B. Hackerson, Scrum at a Fortune 500 manufacturing company, in: Proceedings of Agile Conference, IEEE, 2007, pp. 175–180, doi:10.1109/AGILE.2007.53.
- [18] J. Preece, H. Sharp, Y. Rogers, *Interaction Design: Beyond Human-Computer Interaction*, John Wiley & Sons, 2011.
- [19] M. Renger, G.L. Kolfschoten, G.-J. de Vreede, Challenges in collaborative modeling: a literature review, in: Advances in Enterprise Engineering I, LNBP 10 (2008) 61–77, doi:10.1007/978-3-540-68644-6_5.